

Ex. No: 4 CPU Scheduling Algorithms

1. First Come First Serve (FCFS)

C PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, i;

    int bt[20], wt[20], tat[20];

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);

        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for (i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
```

```

    avg_tat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);

printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

Output:

```

(dhanush@dhanush)-[~]
$ nano fcfs.c

(dhanush@dhanush)-[~]
$ gcc fcfs.c -o fcfs

(dhanush@dhanush)-[~]
$ ./fcfs
Enter Number of Processes: 2
Enter Burst Time for P1: 12
Enter Burst Time for P2: 13

Process BT      WT      TAT
P1      12      0       12
P2      13     12      25

Average Waiting Time = 6.00
Average Turnaround Time = 18.50

```

Shell Program (fcfs.sh)

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```

for ((i=0;i<n;i++))
do
echo "Enter Burst Time for P$(i+1)"
read bt[$i]
done
wt[0]=0
for ((i=1;i<n;i++))
do
wt[$i]=$((wt[i-1]+bt[i-1]))
done
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$(i+1)\t${bt[i]}\t${wt[i]}\t${tat[i]}"

30

done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")

```

echo

echo "Average Waiting Time = \$avg_wt"

echo "Average Turnaround Time = \$avg_tat"

Output:

```
(dhanush@dhanush)-[~]
$ nano fcfs.sh

(dhanush@dhanush)-[~]
$ chmod +x fcfs.sh

(dhanush@dhanush)-[~]
$ bash fcfs.sh
Enter Number of Processes
2
Enter Burst Time for P1
12
Enter Burst Time for P2
13

Process BT      WT      TAT
P1      12      0      12
P2      13     12     25

Average Waiting Time = 6.00
Average Turnaround Time = 18.50
```

2. Shortest Job First (SJF)

C Program:

C Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i,j,temp;
```

```
int bt[20],wt[20],tat[20];
```

```
float avg_wt=0, avg_tat=0;
```

```
printf("Enter Number of Processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
```

```

{
printf("Enter Burst Time for P%d: ",i+1);
scanf("%d",&bt[i]);
}

// Sorting burst times in ascending order
for(i=0;i<n-1;i++)
{
for(j=i+1;j<n;j++)
{
if(bt[i] > bt[j])
{
temp = bt[i];
bt[i] = bt[j];
bt[j] = temp;
}
}
}

```

31

```

}

wt[0] = 0;
for(i=1;i<n;i++)
wt[i] = wt[i-1] + bt[i-1];
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++)
{

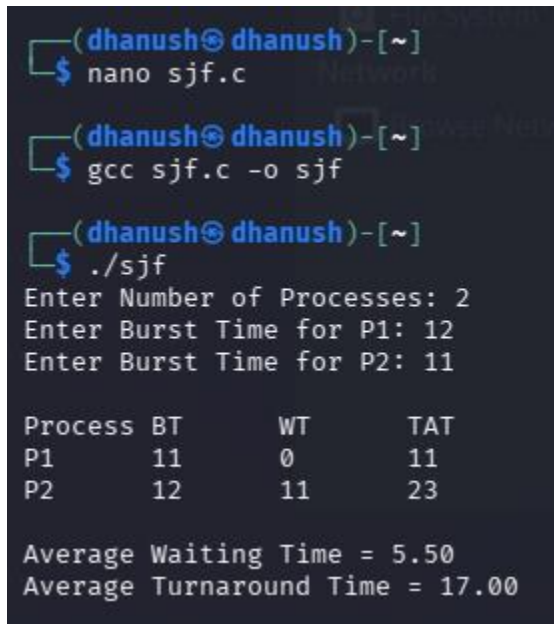
```

```

tat[i] = wt[i] + bt[i];
avg_wt += wt[i];
avg_tat += tat[i];
printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
}

avg_wt = avg_wt / n;
avg_tat = avg_tat / n;
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```



```

(dhanush@dhanush)~$ nano sjf.c
(dhanush@dhanush)~$ gcc sjf.c -o sjf
(dhanush@dhanush)~$ ./sjf
Enter Number of Processes: 2
Enter Burst Time for P1: 12
Enter Burst Time for P2: 11

Process BT      WT      TAT
P1      11      0      11
P2      12      11     23

Average Waiting Time = 5.50
Average Turnaround Time = 17.00

```

Shell Program (sjf.sh)

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0;i<n;i++))
```

```

do
echo "Enter Burst Time for P$((i+1))"
read bt[$i]
done

# Sorting burst times (Ascending Order)
for ((i=0;i<n-1;i++))
do
for ((j=i+1;j<n;j++))
do
if [ ${bt[i]} -gt ${bt[j]} ]
then
temp=${bt[i]}
bt[$i]=${bt[j]}
bt[$j]=$temp
fi
done
done
wt[0]=0
echo
echo -e "Process\tBT\tWT\tTAT"
total_wt=0
total_tat=0
for ((i=0;i<n;i++))
do
if [ $i -ne 0 ]
then

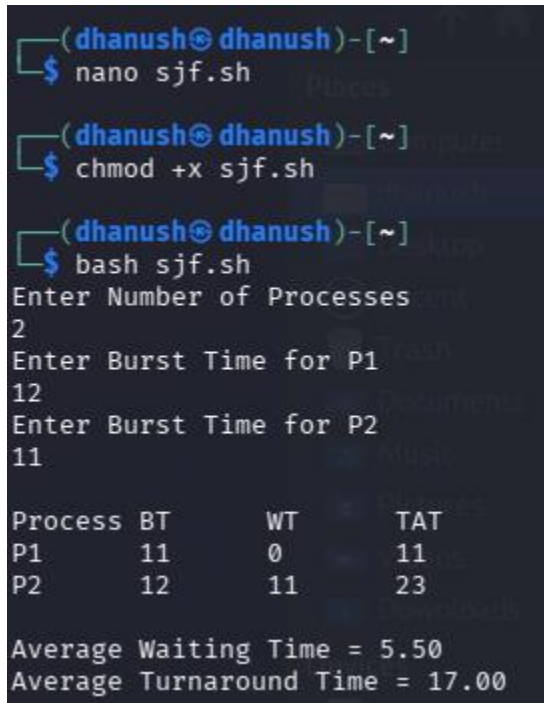
```

```

wt[$i]=$((wt[i-1]+bt[i-1]))
fi
tat[$i]=$((wt[i]+bt[i]))
total_wt=$((total_wt+wt[i]))
total_tat=$((total_tat+tat[i]))
echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done
avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")
echo
echo "Average Waiting Time = $avg_wt"
echo "Average Turnaround Time = $avg_tat"

```

Output:



```

(dhanush@dhanush)-[~]
$ nano sjf.sh
(dhanush@dhanush)-[~]
$ chmod +x sjf.sh
(dhanush@dhanush)-[~]
$ bash sjf.sh
Enter Number of Processes
2
Enter Burst Time for P1
12
Enter Burst Time for P2
11

Process BT      WT      TAT
P1      11      0       11
P2      12      11      23

Average Waiting Time = 5.50
Average Turnaround Time = 17.00

```

3. Priority Scheduling

C Program:


```

#include <stdio.h>

int main()
{
    int n, i, j, temp;

    int bt[20], pr[20], wt[20], tat[20];

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter Burst Time for P%d: ", i + 1);

        scanf("%d", &bt[i]);

        printf("Enter Priority for P%d: ", i + 1);

        scanf("%d", &pr[i]);
    }

    /* Sort according to priority */

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (pr[i] > pr[j])
            {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;

                temp = bt[i];

```

```

        bt[i] = bt[j];

        bt[j] = temp;
    }
}

/* Calculate Waiting Time */

wt[0] = 0;

for (i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

printf("\nProcess\tPriority\tBT\tWT\tTAT\n");

/* Calculate Turnaround Time */

for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];

    avg_wt += wt[i];

    avg_tat += tat[i];

    printf("P%d\t%d\t\t%d\t%d\t%d\n",
           i + 1, pr[i], bt[i], wt[i], tat[i]);
}

avg_wt /= n;

avg_tat /= n;

printf("\nAverage Waiting Time = %.2f", avg_wt);

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

return 0;

```

```
}
```

Output:

```
(dhanush@dhanush)-[~]
$ nano priority.c

(dhanush@dhanush)-[~]
$ gcc priority.c -o priority

(dhanush@dhanush)-[~]
$ ./priority
Enter Number of Processes: 2

Enter Burst Time for P1: 11
Enter Priority for P1: 12

Enter Burst Time for P2: 12
Enter Priority for P2: 11

Process Priority    BT    WT    TAT
P1      11         12     0     12
P2      12         11    12    23

Average Waiting Time = 6.00
Average Turnaround Time = 17.50
```

Shell Script:

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0; i<n; i++))
```

```
do
```

```
    echo "Enter Burst Time for P$((i+1))"
```

```
    read bt[$i]
```

```
    echo "Enter Priority for P$((i+1))"
```

```
    read pr[$i]
```

```
done
```

```

# Sort according to Priority
for ((i=0; i<n-1; i++))
do
    for ((j=i+1; j<n; j++))
    do
        if [ ${pr[i]} -gt ${pr[j]} ]
        then
            temp=${pr[i]}
            pr[$i]=${pr[j]}
            pr[$j]=$temp
            temp=${bt[i]}
            bt[$i]=${bt[j]}
            bt[$j]=$temp
        fi
    done
done

# Calculate Waiting Time
wt[0]=0

echo
echo -e "Process\tPriority\tBT\tWT\tTAT"

total_wt=0
total_tat=0

for ((i=0; i<n; i++))
do
    if [ $i -ne 0 ]
    then

```

```

        wt[$i]=$((wt[i-1] + bt[i-1]))
    fi

    tat[$i]=$((wt[i] + bt[i]))

    total_wt=$((total_wt + wt[i]))

    total_tat=$((total_tat + tat[i]))

    echo -e "P$((i+1))\t${pr[i]}\t${bt[i]}\t${wt[i]}\t${tat[i]}"

done

avg_wt=$(awk "BEGIN {printf \"%.2f\", $total_wt/$n}")
avg_tat=$(awk "BEGIN {printf \"%.2f\", $total_tat/$n}")

echo

echo "Average Waiting Time = $avg_wt"

echo "Average Turnaround Time = $avg_tat"

```

Output:

```

(dhanush@dhanush)-[~]
$ nano priority.sh
(dhanush@dhanush)-[~]
$ chmod +x priority.sh
(dhanush@dhanush)-[~]
$ bash priority.sh
Enter Number of Processes
2
Enter Burst Time for P1
11
Enter Priority for P1
12
Enter Burst Time for P2
12
Enter Priority for P2
11

```

Process	Priority	BT	WT	TAT
P1	11	12	0	12
P2	12	11	12	23

```

Average Waiting Time = 6.00
Average Turnaround Time = 17.50

```

4. Round Robin Scheduling

C Program:

```
#include <stdio.h>

int main()
{
    int n, tq, i;

    int bt[20], rem_bt[20];

    int wt[20] = {0}, tat[20];

    int time = 0, done;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter Number of Processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);

        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");

    scanf("%d", &tq);

    do
    {
        done = 1;

        for (i = 0; i < n; i++)
        {
```

```

    if (rem_bt[i] > 0)
    {
        done = 0;
        if (rem_bt[i] > tq)
        {
            time += tq;
            rem_bt[i] -= tq;
        }
        else
        {
            time += rem_bt[i];
            wt[i] = time - bt[i];
            rem_bt[i] = 0;
        }
    }
}

} while (!done);

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

```

```

    avg_wt /= n;

    avg_tat /= n;

    printf("\nAverage Waiting Time = %.2f", avg_wt);

    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}

```

Output:

```

(dhanush@dhanush)-[~]
$ nano roundrobin.c
(dhanush@dhanush)-[~]
$ gcc roundrobin.c -o roundrobin
(dhanush@dhanush)-[~]
$ ./roundrobin
Enter Number of Processes: 2
Enter Burst Time for P1: 11
Enter Burst Time for P2: 12
Enter Time Quantum: 2

Process BT      WT      TAT
P1      11      10      21
P2      12      11      23

Average Waiting Time = 10.50
Average Turnaround Time = 22.00

```

Shell Program:

```
#!/bin/bash
```

```
echo "Enter Number of Processes"
```

```
read n
```

```
for ((i=0; i<n; i++))
```

```
do
```

```
    echo "Enter Burst Time for P$((i+1))"
```



```
    read bt[$i]
    rem_bt[$i]=${bt[$i]}
done

echo "Enter Time Quantum"
read tq

time=0

while true
do
    done=1

    for ((i=0; i<n; i++))
    do
        if [ ${rem_bt[i]} -gt 0 ]
        then
            done=0

            if [ ${rem_bt[i]} -gt $tq ]
            then
                time=$((time + tq))
                rem_bt[$i]=$((rem_bt[i] - tq))
            else
                time=$((time + rem_bt[i]))
                wt[$i]=$((time - bt[i]))
            fi
        fi
    done
done
```

```

        rem_bt[$i]=0
    fi
fi
done

if [ $done -eq 1 ]
then
    break
fi
done

echo
echo -e "Process\tBT\tWT\tTAT"

total_wt=0
total_tat=0

for ((i=0; i<n; i++))
do
    tat[$i]=$((bt[i] + wt[i]))

    total_wt=$((total_wt + wt[i]))
    total_tat=$((total_tat + tat[i]))

    echo -e "P$((i+1))\t${bt[i]}\t${wt[i]}\t${tat[i]}"
done

```

```
avg_wt=$(awk "BEGIN {printf \"%.2f\\\", $total_wt/$n}")
```

```
avg_tat=$(awk "BEGIN {printf \"%.2f\\\", $total_tat/$n}")
```

```
echo
```

```
echo "Average Waiting Time = $avg_wt"
```

```
echo "Average Turnaround Time = $avg_tat"
```

Output:

```
(dhanush@dhanush)-[~]  
$ nano rr.sh  
  
(dhanush@dhanush)-[~]  
$ chmod +x rr.sh  
  
(dhanush@dhanush)-[~]  
$ bash rr.sh  
Enter Number of Processes  
2  
Enter Burst Time for P1  
11  
Enter Burst Time for P2  
12  
Enter Time Quantum  
2  
  
Process BT      WT      TAT  
P1      11      10      21  
P2      12      11      23  
  
Average Waiting Time = 10.50  
Average Turnaround Time = 22.00
```

